

From Tokens to Numbers

Text Representation, Word Embeddings, and Linear Models

Yasser Hifny

Faculty of Computers and Artificial Intelligence
Capital University

1. From Text to Tokens

The NLP Representation Pipeline

Text $\rightarrow$ Tokens $\rightarrow$ Numerical Representation $\rightarrow$ Model

Consider:

The cat sits on the mat

Different tokenization strategies produce different units.

Tokenization	Example	Main Idea
Word	The cat sits on the mat	One token per word
Character	T h e ...	One token per character
BPE	The ca t sits on the mat	Subword units

Tokenization: Word, Character, and BPE

Word Tokenization

Simple and interpretable, but produces a large vocabulary and can suffer from out-of-vocabulary words.

Character Tokenization

Uses a very small vocabulary and can represent any word, but creates very long sequences.

BPE — Byte Pair Encoding

Builds subword units by repeatedly merging frequent symbol pairs.

playing $\rightarrow$ play + ing

Key Idea

Tokenization determines the basic units from which numerical representations are constructed.

2. Bag of Words (BOW)

Basic Idea

Represent a document by counting how many times each vocabulary word occurs.

Consider three documents:

D_1 : the cat likes milk

D_2 : the dog likes milk

D_3 : the cat likes fish

Vocabulary:

$V = [\text{the, cat, dog, likes, milk, fish}]$

BOW Numerical Representation

Using the vocabulary

[the, cat, dog, likes, milk, fish]

we obtain:

	the	cat	dog	likes	milk	fish
D_1	1	1	0	1	1	0
D_2	1	0	1	1	1	0
D_3	1	1	0	1	0	1

Therefore:

$$D_1 = [1, 1, 0, 1, 1, 0]$$

Characteristics

- Simple numerical representation.
- Captures word frequency.
- Ignores word order.

BOW: Strengths and Limitations

Advantages

- Simple.
- Fast to compute.
- Easy to interpret.
- Effective for classical classifiers.

Limitations

- Ignores word order.
- Ignores semantic similarity.
- Very sparse vectors.
- Large vocabulary $\Rightarrow$ large dimensions.

For example:

dog bites man

and

man bites dog

have the same BOW representation.

BOW does not represent word order.

3. TF-IDF Representation

Term Frequency

TF measures how frequently a term occurs in a document.

$$TF(t, d) = \frac{\text{number of occurrences of } t \text{ in } d}{\text{total number of terms in } d}$$

Inverse Document Frequency

$$IDF(t) = \log \frac{N}{df(t)}$$

where:

- N = number of documents
- $df(t)$ = number of documents containing term t

Intuition

Common words receive lower importance (i.e stop words), while words that are specific to a document receive higher importance.

TF-IDF Example

Suppose there are:

$$N = 1000$$

documents.

For the word cat:

$$df(\text{cat}) = 100$$

Then:

$$IDF(\text{cat}) = \log \frac{1000}{100} = \log(10)$$

For the word the:

$$df(\text{the}) = 900$$

Therefore:

$$IDF(\text{the}) = \log \frac{1000}{900}$$

4. From BOW to Word Embeddings

BOW

A word is represented by its position in the vocabulary.

For example:

$$\text{cat} \rightarrow [0, 1, 0, 0, 0, \dots, 0]$$

Word Embedding

A word is represented by a dense, low-dimensional vector:

$$\text{cat} \rightarrow [0.21, -0.43, 0.72, 0.18, \dots]$$

Key Difference

BOW represents words using discrete vocabulary positions; embeddings represent words using learned numerical features.

CBOW and Skip-Gram

CBOW — Continuous Bag of Words

Predict the target word from its surrounding context.

context → target

Example:

The cat ___ on the mat

Predict:

sits

Skip-Gram

Predict surrounding context words from the target word.

target → context

CBOW and Skip-Gram NN

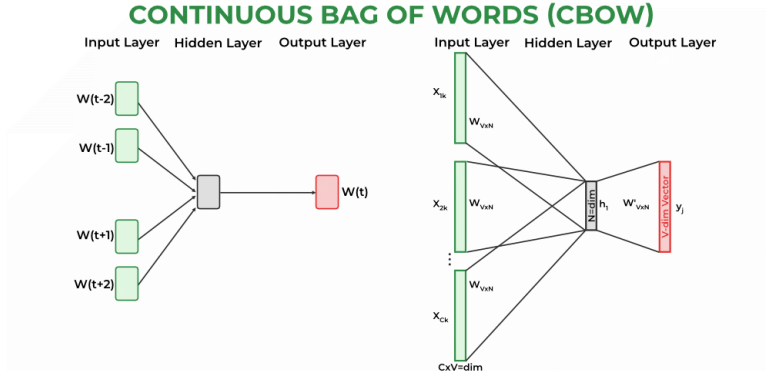


Figure: CBOW NN.

<https://www.geeksforgeeks.org/nlp/word-embeddings-in-nlp-comparison-between-cbow-and-skip-gram-models/>

CBOW and Skip-Gram NN

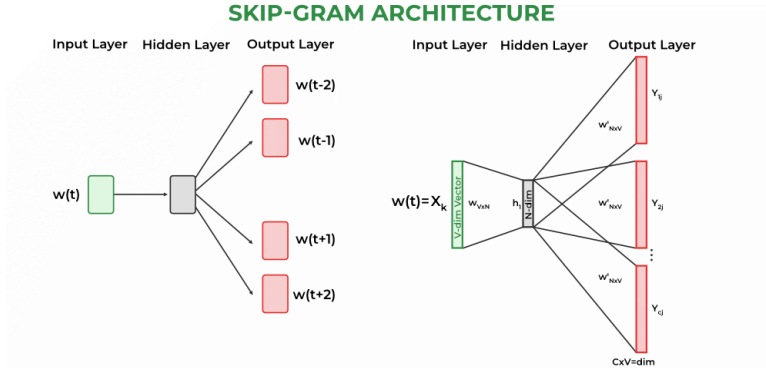


Figure: Skip-Gram NN.

<https://www.geeksforgeeks.org/nlp/word-embeddings-in-nlp-comparison-between-cbow-and-skip-gram-models/>

Learning Word Embeddings

Suppose the vocabulary is:

$$V = \{\text{king}, \text{queen}, \text{man}, \text{woman}, \dots\}$$

The model learns a vector for each word:

$$\mathbf{e}_{\text{king}}, \mathbf{e}_{\text{queen}}, \mathbf{e}_{\text{man}}, \mathbf{e}_{\text{woman}} \in \mathbb{R}^d$$

The vectors are learned from word-context relationships.

Distributional Hypothesis

Words that occur in similar contexts tend to have similar representations.

You shall know a word by the company it keeps.

5. Word Embedding Analogy

A famous property of word embeddings is that some semantic relationships can be represented as vector differences.

Consider:

king $\leftrightarrow$ queen

and

man $\leftrightarrow$ woman

The relationship can approximately be represented as:

$$\mathbf{e}_{queen} - \mathbf{e}_{king} \approx \mathbf{e}_{woman} - \mathbf{e}_{man}$$

Rearranging:

$$\mathbf{e}_{queen} - \mathbf{e}_{king} + \mathbf{e}_{man} \approx \mathbf{e}_{woman}$$

Queen is to King as Woman is to Man

A small illustrative example:

$$\mathbf{e}_{king} = [0.8, 0.2, 0.7]$$

$$\mathbf{e}_{queen} = [0.8, 0.8, 0.7]$$

$$\mathbf{e}_{man} = [0.3, 0.2, 0.4]$$

Then:

$$\mathbf{e}_{queen} - \mathbf{e}_{king} = [0, 0.6, 0]$$

Adding this relationship to the vector for **man**:

$$[0.3, 0.2, 0.4] + [0, 0.6, 0] = [0.3, 0.8, 0.4]$$

which is assumed to be close to:

$\mathbf{e}_{woman}$

One-hot encoding

		cat	mat	on	sat	the
the	=>	0	0	0	0	1
cat	=>	1	0	0	0	0
sat	=>	0	0	0	1	0

A 4-dimensional embedding

cat =>

1.2

-0.1

4.3

3.2

mat =>

0.4

2.5

-0.9

0.5

on =>

2.1

0.3

0.1

0.4

...

...

6. Keras Example: Embedding Layer

```
import tensorflow as tf
from tensorflow.keras import layers

vocab_size = 10000
embedding_dim = 128

model = tf.keras.Sequential([
    layers.Embedding(
        input_dim=vocab_size,
        output_dim=embedding_dim
    ),
    layers.GlobalAveragePooling1D(),
    layers.Dense(1, activation="sigmoid")
])

model.compile(
    optimizer="adam",
    loss="binary_crossentropy",
    metrics=["accuracy"]
)
```

What happens?

Each integer token ID is mapped to a trainable vector of dimension 128.

Keras Embedding: Input and Output

Suppose:

$$\text{cat} \rightarrow 25$$

and:

$$\text{dog} \rightarrow 81$$

The embedding matrix is:

$$E \in \mathbb{R}^{10000 \times 128}$$

Then:

$$E[25] = \mathbf{e}_{cat}$$

and

$$E[81] = \mathbf{e}_{dog}$$

Thus:

7. Embedding Layer: Keras vs. PyTorch

Keras

```
layers.Embedding(  
    input_dim=10000,  
    output_dim=128  
)
```

The layer creates:

$$E \in \mathbb{R}^{10000 \times 128}$$

PyTorch

```
nn.Embedding(  
    num_embeddings=10000,  
    embedding_dim=128  
)
```

The layer creates:

$$E \in \mathbb{R}^{10000 \times 128}$$

Conceptually

Both layers implement the same basic operation:

token ID $\rightarrow$ row of embedding matrix

Embedding Layer: Training

Let:

$$E \in \mathbb{R}^{V \times d}$$

where:

- V = vocabulary size
- d = embedding dimension

For token i :

$$\mathbf{e}_i = E_{i,:}$$

During training, gradients update the selected embedding vectors:

$$E_{i,:} \leftarrow E_{i,:} - \eta \frac{\partial L}{\partial E_{i,:}}$$

Important

The embedding is not necessarily a fixed representation. It can be learned jointly with the downstream task.

8. Linear Models: Binary Classification

A document can be represented by a vector:

$$\mathbf{x} \in \mathbb{R}^d$$

A linear model computes:

$$z = \mathbf{w}^T \mathbf{x} + b$$

For binary classification:

$$\hat{y} = \sigma(z) = \frac{1}{1 + e^{-z}}$$

For example:

$$\hat{y} > 0.5 \Rightarrow \text{Class 1}$$

$$\hat{y} \leq 0.5 \Rightarrow \text{Class 0}$$

Example

Multiclass Classification

Suppose the document belongs to exactly one of:

$\{\text{Sports, Politics, Economy, Technology}\}$

The linear model produces:

$$\mathbf{z} = W\mathbf{x} + \mathbf{b}$$

where:

$$W \in \mathbb{R}^{C \times d}$$

Softmax converts logits into probabilities:

$$P(y = c | \mathbf{x}) = \frac{e^{z_c}}{\sum_{j=1}^C e^{z_j}}$$

Decision

Select the class with the largest probability.

Multilabel Classification

In multilabel classification, a document can belong to multiple classes simultaneously.

Example:

Document $\rightarrow$ {Politics, Economy, International}

The model produces one probability per label:

$$\hat{\mathbf{y}} = [0.91, 0.82, 0.13, 0.76]$$

Each output uses an independent sigmoid:

$$\hat{y}_j = \sigma(z_j)$$

Difference from Multiclass

Multiclass: one class

Multilabel: zero, one, or several classes

9. Assignment: Sanad Document Classification

Objective

Build a document classifier for the **SANAD Arabic news dataset** using word embeddings and PyTorch.

- 1 Load and preprocess the Arabic documents.
- 2 Tokenize the documents.
- 3 Build a vocabulary.
- 4 Convert words into integer token IDs.
- 5 Use a PyTorch embedding layer:

`nn.Embedding`

- 6 Construct a document representation by aggregating word embeddings.
- 7 Train a linear classification layer.
- 8 Evaluate the model using:
 - Accuracy
 - Precision
 - Recall
 - F1-score

Assignment: Suggested Architecture

Arabic Document



Tokenizer



Token IDs



Embedding



Any Questions?

From Tokens to Numbers

Tokenization → BOW/TF-IDF → Embeddings → Classification